THE PROJECT LIFE CYCLE AND SYSTEM DEFINITION

A. Module Overview

As Computer Engineers, you will rarely work on isolated tasks. You will be designing embedded systems, deploying enterprise networks, or developing complex software applications. These are *projects*, temporary, unique endeavors constrained by time, cost, and performance requirements. This module introduces the **Systems Development Cycle (SDC)** and the **Project Life Cycle**, providing the framework for how systems are conceived, defined, built, and operated. Week 10 plunges into the mathematical and analytical tools used to plan and control these projects, specifically the **Work Breakdown Structure (WBS)**, **Critical Path Method (CPM)**, and **PERT**.

B. Learning Outcomes

By the end of this module segment, learners will be able to:

1. **Analyze** the four phases of the Systems Development Cycle (SDC) and apply them to hardware/software development scenarios.
2. **Formulate** clear, traceable system requirements and specifications from vague user needs.
3. **Design** a comprehensive Work Breakdown Structure (WBS) for a complex Computer Engineering project.
4. **Calculate** project durations, early/late start and finish times, and total slack using Activity-on-Node (AON) network diagrams.
5. **Evaluate** project schedule risks using the Program Evaluation and Review Technique (PERT) and formulate time-cost tradeoff strategies (crashing).

I. Diagnostic Pre-Assessment

Instructions: Test your baseline knowledge before beginning the lesson. Check your answers at the end of the Week 8 section.

1. Which phase of the Systems Development Cycle (SDC) is characterized by the highest ability to influence the overall life-cycle cost of a system?
2. **True or False:** A "User Requirement" and a "System Specification" are the exact same thing in project management.
3. In system implementation, what is a "Cold Turkey" (Big Bang) conversion strategy, and why is it risky for IT systems?

II. Conceptual Foundations: The Systems Development Cycle (SDC)

According to Nicholas & Steyn (2021), every human-made system—whether it is a new microarchitecture, a nationwide 5G network, or a robotic assembly line—follows a logical sequence of development called the **Systems Development Cycle (SDC)**. The SDC consists of four distinct phases: Conception, Definition, Execution, and Operation. The first three phases constitute the **Project Life Cycle**.

Understanding this cycle is critical because decisions made in the early phases lock in the majority of the project's costs and determine its ultimate success or failure.

Phase A: Conception (Project Initiation and Feasibility)

The conception phase begins when a customer, user, or sponsor recognizes a need, a problem, or an opportunity.

1. Initial Investigation: This is a "quick and dirty" assessment. A need is identified (e.g., "Our university campus Wi-Fi is too slow and has dead zones"). The organization performs a preliminary study to determine if the problem is worth solving. If it aligns with organizational goals, it moves to the next step.

2. Project Feasibility Study: If the initial idea survives, a formal feasibility study is conducted. This study rigorously evaluates alternative solutions based on:

- *Technical Feasibility:* Does the technology exist to solve this? Do we possess the engineering talent?
- *Economic Feasibility:* Will the benefits (ROI) outweigh the costs?
- *Operational Feasibility:* Will the end-users actually adopt and use the system?

3. The Request for Proposal (RFP) and Proposal Generation: If the customer lacks the internal resources to build the system, they outsource it. They publish an **RFP**, a formal document sent to potential contractors. A good RFP contains the Statement of Work (SOW), expected deliverables, timeline, and proposal evaluation criteria. Contractors review the RFP and, if they believe they can technically and profitably deliver the system, they submit a **Proposal**. The proposal is a sales document and a preliminary project plan rolled into one. It outlines the contractor's technical approach, project management plan, and cost estimate.

Phase B: Definition (System and Project Definition)

Once a proposal is accepted and a contract is signed, the Definition phase begins. Nicholas & Steyn (2021) refer to this as "front-end loading." The goal here is to establish *exactly* what will be built and *how* the project will be managed.

1. System Definition and Requirements Analysis: This is where many tech projects fail. Customers speak in **User Requirements** (vague, operational needs like "The app must be secure

and load instantly"). Engineers cannot build from vague statements. The project team must translate these into **System Requirements** (what the system must do to meet the user need) and finally into **System Specifications** (highly detailed, measurable, technical criteria).

- *User Requirement:* "The drone must fly long enough to map a 50-acre farm."
- *System Requirement:* "The drone must have a continuous flight time of at least 45 minutes carrying a 2kg payload."
- *System Specification:* "The drone power subsystem will utilize a 4S 14.8V 5000mAh Lithium-Polymer battery with a maximum discharge rate of 30C."

To ensure no features are missed and no unnecessary features are added (preventing "scope creep"), engineers use a **Requirements Traceability Matrix**. This document maps every technical specification back to a specific user requirement.

2. Project Definition: Parallel to defining the system, the project manager defines the project. This involves creating the detailed execution plan, establishing the final budget, and setting the baseline schedule.

Phase C: Execution (Acquisition)

This is the longest and most capital-intensive phase of the project life cycle. The plans made in Phase B are put into action.

1. **Detail Design:** Engineers create schematics, PCB layouts, and software architectures. Prototypes are built and tested.
2. **Production/Build:** The system is fabricated or coded.
3. **Implementation (System Conversion):** The system is installed in the user's environment. For IT and CpE projects, Nicholas & Steyn identify three primary conversion strategies:
    - *Parallel Installation:* The old system and new system run simultaneously until the new system is proven. (High cost, low risk).
    - *Pilot Operation:* The new system is rolled out to a small fraction of users first. (Moderate cost, moderate risk).
    - *Cold Turkey (Big Bang):* The old system is turned off, and the new system is turned on immediately. (Low cost, extremely high risk).
4. **Closeout:** The project is handed over to the user. A crucial, often overlooked step is the **Post-Completion Project Review** (Postmortem), where the team documents "lessons learned" to improve future projects.

Phase D: Operation

The project is officially over. The system is now operational and in the hands of the end-users. The original project team is disbanded, though a separate maintenance or support team may take over to handle upgrades and repairs until the system reaches the end of its useful life and is disposed of.

III. Computer Engineering Contextualization: Waterfall vs. Agile in the SDC

The standard four-phase SDC heavily implies a "Waterfall" methodology: you finish Phase B (Definition) completely before starting Phase C (Execution). For building a bridge or a skyscraper, this is mandatory. You cannot pour a concrete foundation while still deciding how many floors the building will have.

However, in software engineering and embedded systems design, requirements change rapidly. If a CpE team spends a year in Phase B defining software specifications, the technology or market may have changed by the time they start Phase C.

Therefore, modern Engineering Management adapts the SDC using **Rapid Prototyping** and **Iterative (Agile) Development**. In this adapted model, the team moves quickly from a high-level Phase B into a mini-Phase C to build a small, functional prototype. They show this to the user, gather feedback, and cycle back to Phase B to refine the requirements. This ensures the final system actually meets the user's needs, even if those needs evolve during the project.

IV. Guided Practice: Requirements Traceability

Learning Outcome: Formulate system specifications from vague user requirements.

Scenario: You are the Lead Systems Engineer for a startup developing a "Smart Health Wearable" for elderly patients. The marketing department provides you with the following User Requirement gathered from a focus group: *"The device needs to monitor my heart, last a long time without charging, and automatically call for help if I fall down in the shower."*

Task: Break this down into distinct User Requirements, translate them into System Requirements, and finally into technical System Specifications.

Step-by-Step Breakdown:

User Requirement (The "What")	**System Requirement (The "How" functionally)**	**System Specification (The "How" technically)**
UR1: Monitor my heart.	SR1: The system must continuously monitor heart rate and detect abnormal arrhythmias.	SS1: Integrate an optical photoplethysmography (PPG) sensor sampling at 50Hz, linked to an onboard anomaly-detection microcontroller algorithm.

User Requirement (The "What")	**System Requirement (The "How" functionally)**	**System Specification (The "How" technically)**
UR2: Lasts a long time without charging.	SR2: The device must operate continuously for at least 7 days on a single charge.	SS2: Utilize a 3.7V 300mAh Li-ion coin cell battery. Total system power draw in active mode must not exceed 1.5mA.
UR3: Automatically call for help if I fall in the shower.	SR3a: The device must detect a sudden fall and impact. SR3b: The device must be highly water-resistant.	SS3a: Integrate a 3-axis MEMS accelerometer detecting impact forces > 2.5G followed by 10 seconds of immobility. SS3b: Device casing must achieve an IP68 ingress protection rating (submersible to 1.5 meters for 30 minutes).

Self-Reflection Question: Why is SS3b (IP68 rating) directly traceable to the user requirement "fall in the shower"? What would happen if the engineering team only designed the fall detection (SS3a) but forgot the waterproofing?

V. Week 8 Chapter-End Assessment

Part I: Analytical Questions

1. Explain the "Environmental Fallacy" in the context of systems development. If a CpE team designs a high-performance server farm without considering the cooling capacity of the client's building, which SDC phase was flawed, and why?

2. Contrast the "Pilot Operation" and "Cold Turkey" conversion strategies. Provide a specific IT deployment scenario where Cold Turkey would be absolutely unacceptable, and justify your answer.

Part II: Application Task You are managing the deployment of an automated toll-collection system for a major expressway. Write one complete traceability chain (User Requirement → System Requirement → System Specification) addressing the issue of vehicles passing through the toll gate at high speeds at night.